ELL8299 Assignment 1: Fine-Tune, Sanitize, Extract

Uthith Naraayanan G — 2025JRB2054 — ELL8299

Adapter access: Google Drive adapter folder

1. Part A: PEFT Sweep

I swept LoRA placement, rank/scaling, and quantization on Qwen2.5-1.5B-Instruct. Table 1 reports the exact validation and efficiency measurements. A4 was selected as the sanitizer because it achieved the best validation entity F1 (0.9358) and the lowest validation leak rate (0.03579) among the six runs.

Table 1: Part A sweep. F1 and leak are validation metrics.

Run	Target	r	α	Quant.	Train.%	F1	Leak
A1	q,v	8	16	–	0.071	.8818	.0410
A2	q,k,v,o	8	16	–	0.141	.8990	.0376
A3	all 7	8	16	–	0.595	.9186	.0366
A4	**all 7**	**32**	**64**	–	**2.336**	**.9358**	**.0358**
A5	all 7	32	16	–	2.336	.9273	.0361
A6	all 7	32	64	4-bit	2.336	.9178	.0360

Placement. Expanding from only q/v (A1) to attention projections (A2), then to all seven attention+MLP projections (A3), raised F1 from 0.8818 to 0.9186 while reducing leakage from 4.10% to 3.66%. The cost was substantial: throughput fell from 5895 to 1481 tokens/s as more LoRA modules were trained. This indicates that MLP adaptation mattered for the structured masking task, not only attention adaptation.

Rank and scaling. With the same seven-module placement, increasing from $r = 8, \alpha = 16$ (A3) to $r = 32, \alpha = 64$ (A4) added 1.72 F1 points and reduced leak by 0.080 percentage points. Holding $r = 32$ fixed, A4 also beat A5 ($\alpha = 16$) by 0.85 F1 points, suggesting that the stronger LoRA scaling was beneficial at this rank. QLoRA A6 reduced peak memory from 9.48 to 8.07 GB (14.9%) and training time by 47.2%, but lost 1.80 F1 points relative to A4. I therefore favored sanitizer quality over the memory/speed gain.

2. Part B: Sanitization Quality

The selected A4 sanitizer was evaluated deterministically on 2,000 in-distribution (ID) examples and 1,000 OOD examples. Table 2 compares the available systems. The fine-tuned model dominates regex and zero-shot on ID data: exact micro-F1 is 0.9376 and $\text{IoU}_{0.5}$ micro-F1 is 0.9411, with a 3.65% leak rate. Regex is fast but misses most entity classes; zero-shot masking is both inaccurate and heavily over-masking.

Table 2: Part B metrics. OOD is the fine-tuned sanitizer on the OOD set.

System	Exact F1	IoU F1	Leak	Overmask	Lat. ms
Fine-tuned ID	.9376	.9411	.0365	.0032	532.0
Regex ID	.2105	.2225	.7691	.0028	0.18
Zero-shot ID	.0527	.0577	.5815	.2013	631.0
Fine-tuned OOD	.1814	.2017	.3393	.0018	235.8

The OOD result is the main weakness: exact F1 drops by 80.6% relative to ID, while leakage increases by a factor of 9.30. Several OOD classes have near-zero exact F1 (FIRSTNAME, LASTNAME, PREFIX, AMOUNT, JOBAREA, PASSWORD), whereas DATE (0.779) and EMAIL (0.601 exact; 0.860 overlap) transfer better. This shows that the model learned label/format regularities that do not fully generalize to changed entity distributions and schemas.

Error taxonomy. On ID data there were 643 missed gold entities at IoU$\geq$ 0.5, 876 spurious predictions, 45 boundary/near-miss entities, 466 leaked gold values, and 411 alignment failures. On OOD data these became 2704 misses, 1625 spurious predictions, 55 boundary errors, 1103 leaked values, and 37 alignment failures. Thus the dominant OOD failure is under-detection rather than span alignment. The regex baseline leaked 9828 gold values; zero-shot leaked 7431 and also produced 5855 alignment failures.

Limitation. A Presidio result was not present in the archived run outputs used to prepare this report, so I do not invent a score for it. This is an experimental gap that should be closed in a rerun if time permits.

3. Part C: Memorization and Extraction

V-raw and V-san used the same victim-model training setup; the only intended change was the corpus text. Token-weighted held-out perplexity was 14.19 for V-base, 8.20 for V-raw, and 12.84 for V-san. Raw fine-tuning therefore improved perplexity by 42.2% relative to the base model, while sanitization retained only a 9.5% improvement; V-san is 56.5% worse in perplexity than V-raw. This is the measured utility cost of sanitizing before victim training.

Canaries and targeted PII. V-raw is strongly memorizing: mean canary exposure is 13.13, with the sampled estimator saturating at 13.2877 for repetition counts 4, 16 and 64. V-san reduces mean exposure to 1.27 (90.3% below V-raw) and has no monotonic growth with repetition. The sampled-rank estimator uses only 10,000 candidates to extrapolate to a one-million-secret space, so rank 1 yields an exposure ceiling; high V-raw values are therefore resolution-limited, not exact ranks in the full space. Targeted member leakage also falls sharply: greedy recovery drops from 39.5% to 1.0% and sampled-any10 from 43.0% to 2.5% after sanitization.

Untargeted extraction. PII recovery falls from 59/194 (30.4%) for V-raw to 12/194 (6.19%) for V-san, a 79.7% reduction. However, the membership-ranking metrics are not uniformly improved: V-san's zlib-normalized P@100 is 0.89, higher than V-raw's 0.44. Hence sanitization strongly reduces direct PII recovery but does not make every membership score less separable. The reported PII recovery is additionally limited by the released bundle: the expected PII-index file was absent, so evaluation used the 200 known member values from `partC_targeted.jsonl` (194 distinct

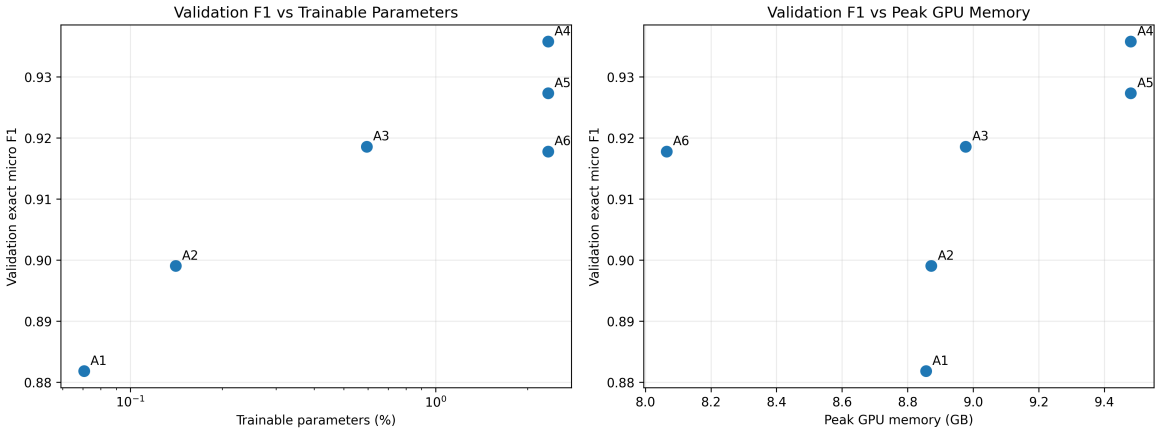


Figure 1: Part A efficiency/quality trade-off. A4 gives the highest validation F1, while A6 is the most memory-efficient high-rank configuration.

Table 3: Part C comparison. Targeted entries are member/control. P@100 is reported in the order PPL / reference ratio / lowercase ratio / zlib-normalized.

Metric	V-base	V-raw	V-san
Held-out PPL	14.19	8.20	12.84
Discoverable exact-match	.0012	.0302	.1960
Targeted greedy leak (member/control)	.010/.025	.395/.045	.010/.040
Targeted sampled-any10 (member/control)	.015/.075	.430/.105	.025/.045
Mean canary exposure	2.32	13.13	1.27
Untargeted P@100 (4 scores)	.01/.01/.01/.01	.33/.36/.12/.44	.42/.35/.14/.89
Untargeted PII recovery	.103	.304	.0619
Recovered distinct PII values	20/194	59/194	12/194

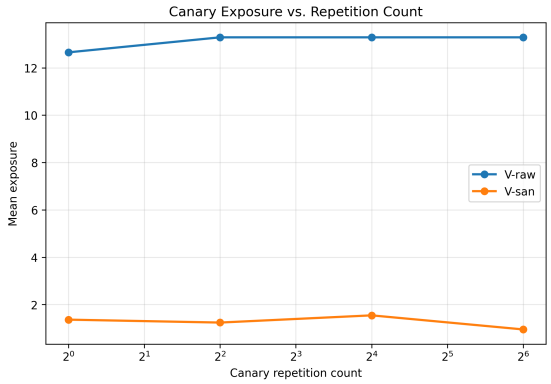


Figure 2: Mean canary exposure versus repetition count. V-raw rapidly reaches the sampled-rank ceiling, while V-san stays near 1–1.5 bits.

normalized values).

Unexpected discoverable result. V-san has a higher prefix-to-suffix exact-match rate (19.6%) than V-raw (3.02%), reaching 100% at repetition counts 8 and 32. This does not imply more raw PII leakage: the V-san attack evaluates sanitized training text, where repeated placeholder/canonicalized patterns are easier to predict exactly. The contradiction between high sanitized-text memorization and low PII/canary recovery is why I treat the four attacks as complementary rather than interchangeable privacy metrics.

Two traced leaks (redacted).

(1) V-raw targeted member `doc_00022`, label LASTNAME1: `Li***ni`; greedy recovery succeeded.

(2) V-san targeted member `doc_00004`, label DATE: `20***24`; greedy failed but at least one of 10 sampled continuations recovered the value. This surviving date-like value is a concrete sanitizer miss that propagated into V-san.

All traces are redacted; no raw extraction dump is included.

4. What I Would Build Next

The results suggest a **defense-in-depth sanitizer** rather than simply increasing LoRA capacity. First, I would add a high-recall deterministic layer for structured entities (dates, identifiers, emails, phones and number patterns) before the learned sanitizer. Second, I would train the learned sanitizer with OOD augmentation and hard-negative mining focused on the classes that collapsed under distribution shift. Third, I would add a post-sanitization verifier: re-run an independent detector and reject/re-mask any text with residual PII. Training data would be admitted only after passing this gate.

For privacy validation, I would make extraction tests part of the training-data release pipeline: targeted leak rate, untargeted PII recovery, and canary exposure would be acceptance metrics alongside F1. I would also avoid replacing many entities with a single repetitive template; randomized type-preserving placeholders or span deletion could reduce the strong discoverable memorization seen in V-san while preserving syntax. Finally, if utility permits, I would test privacy-aware training (e.g., clipping/noising or deduplication of repeated documents) after sanitization, because V-raw shows that repetition quickly drives canary exposure to the estimator ceiling.

Honour Code and Disclosure

"Even though I have taken help from the following students and LLMs in terms of discussing ideas and coding practices, all my code is written by me."

Students consulted/discussed: Tarun Kumar.
LLMs/tools used: ChatGPT (OpenAI) for debugging, formatting, and drafting assistance; Kaggle notebooks for GPU experiments.
Reproducibility note: seed 42 was used for the reported experiments. All numerical claims in this draft were taken from the submitted result artifacts; no missing baseline value was fabricated.